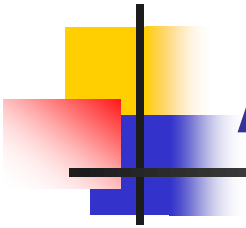
Projeto e Análise de Algoritmos

Algoritmos Gulosos e Grafos (Parte II dos Gulosos)

Universidade Federal do Amazonas
Instituto de Computação



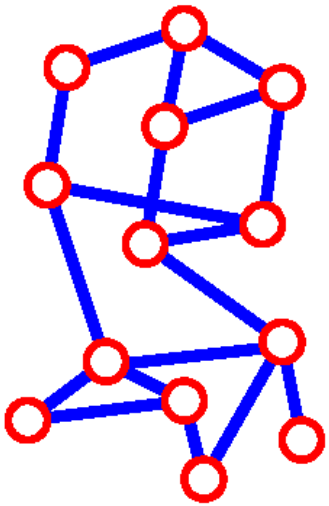


Algoritmos Gulosos

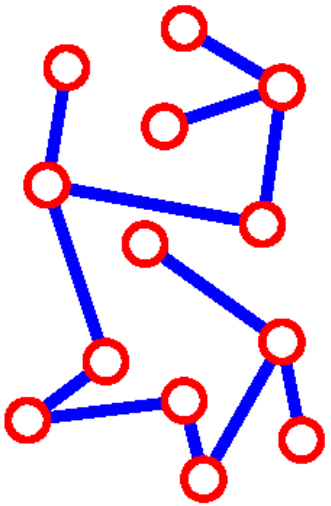
- Utilizam o mesmo conceito de subestrutura ótima usado na prog. Dinâmica.
- Fazem escolhas com base em dados locais para encontrar boas soluções
- Técnica pode ser usada para obter resultados aproximados em alguns casos

Árvore Espalhada

- Uma árvore espalhada de um grafo G é um subgrafo de G que
  - é uma árvore
  - Contém todos os vértices de G



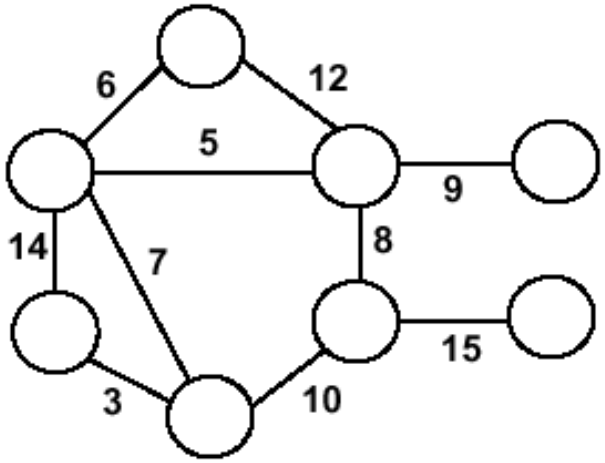
G



Árvore espalhada de G

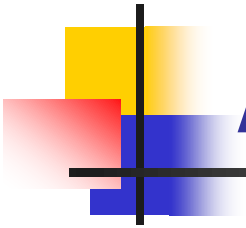
Árvore espalhada mínima

- *Seja G* = (*V,E*) um grafo conexo e não dirigido
- Seja uma função de custo *W*: $E \rightarrow R$ *que atribui um custo a cada aresta de G*



- Árvore Espalhada: Uma árvore que conecta todos os vértices
- Árvores Espalhada Mínima: árvore espalhada que minimiza:

$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

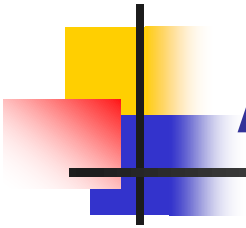


Aplicações

- Clasificação e Agrupamento:
  - Juszczak, P., Tax, D. M., Pe, E., & Duin, R. P. (2009). **Minimum spanning tree based one-class classifier**. *Neurocomputing*, *72*(7-9), 1859-1869.
  - GRYGORASH, Oleksandr; ZHOU, Yan; JORGENSEN, Zach. Minimum spanning tree based clustering algorithms. In:**(ICTAI'06)**. IEEE, 2006. p. 73-81.
- Qualidade de experimentos:
  - FRANCO, J. et al. Minimum Spanning Tree: A new approach to assess the quality of the design of computer experiments. **Chemometrics and intelligent laboratory systems**, v. 97, n. 2, p. 164-169, 2009.
- Segmentação de Imagens:
  - XU, Ying; UBERBACHER, Edward C. 2D image segmentation using minimum spanning trees. **Image and Vision Computing**, v. 15, n. 1, p. 47-57, 1997.

Aplicações

- Clasificação e Agrupamento:
  - Juszczak, P., Tax, D. M., Pe, E., & Duin, R. P. (2009). **Minimum spanning tree based one-class classifier**. *Neurocomputing*, *72*(7-9), 1859-1869.
  - GRYGORASH, Oleksandr; ZHOU, Yan; JORGENSEN, Zach. Minimum spanning tree based clustering algorithms. In:**(ICTAI'06)**. IEEE, 2006. p. 73-81.
- Qualidade de experimentos:
  - FRANCO, J. et al. Minimum Spanning Tree: A new approach to assess the quality of the design of computer experiments. **Chemometrics and intelligent laboratory systems**, v. 97, n. 2, p. 164-169, 2009.
- Segmentação de Imagens:
  - XU, Ying; UBERBACHER, Edward C. 2D image segmentation using minimum spanning trees. **Image and Vision Computing**, v. 15, n. 1, p. 47-57, 1997.



Aplicações

- Redes Sociais:
  - Lappas, Theodoros, Kun Liu, and Evimaria Terzi. "Finding a team of experts in social networks." *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 2009.
- Bioinformática:
  - Xu, Ying, Victor Olman, and Dong Xu. "Clustering gene expression data using a graph-theoretic approach: an application of minimum spanning trees." *Bioinformatics* 18.4 (2002): 536-545.

Projeto de redes (das mais diversas coisas)



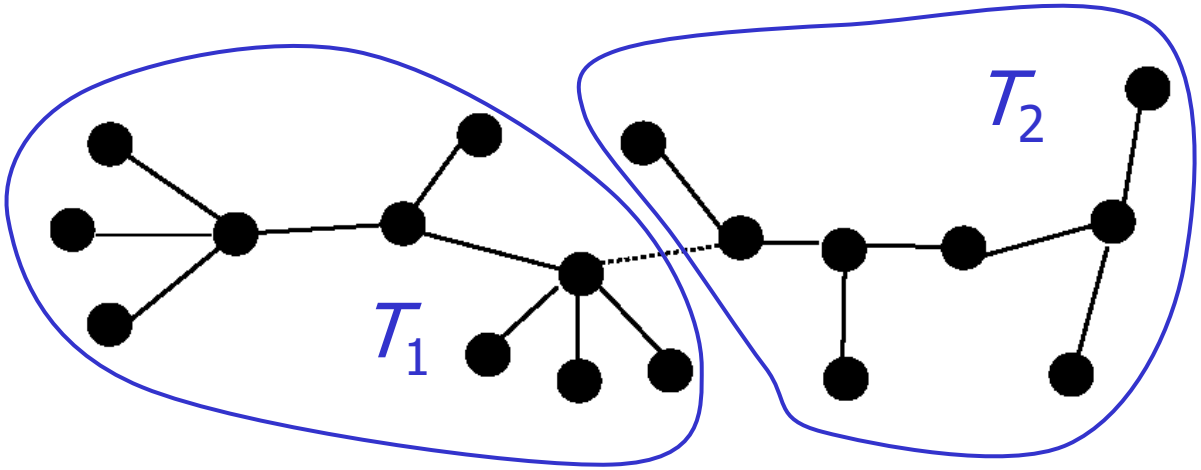


Aproximação de problemas de otimização complicados

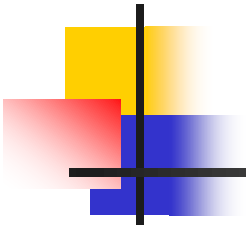
- Árovre de Steiner
  - Achar uma árvore mínima que liga S vértices de um grafo G=(V,E), com S contido em V. Note que se S=V, vira a AEM
- Problema do Caixeiro viajante(PCV)
  - Determinar a menor rota para percorrer uma série de cidades (visitando uma única vez cada uma delas), retornando à cidade de origem.
  - Troque cidade por vértice e coneções entre cidades por arestas pra modelar em um grafo!

Sub-estrutura ótima

- AEM T



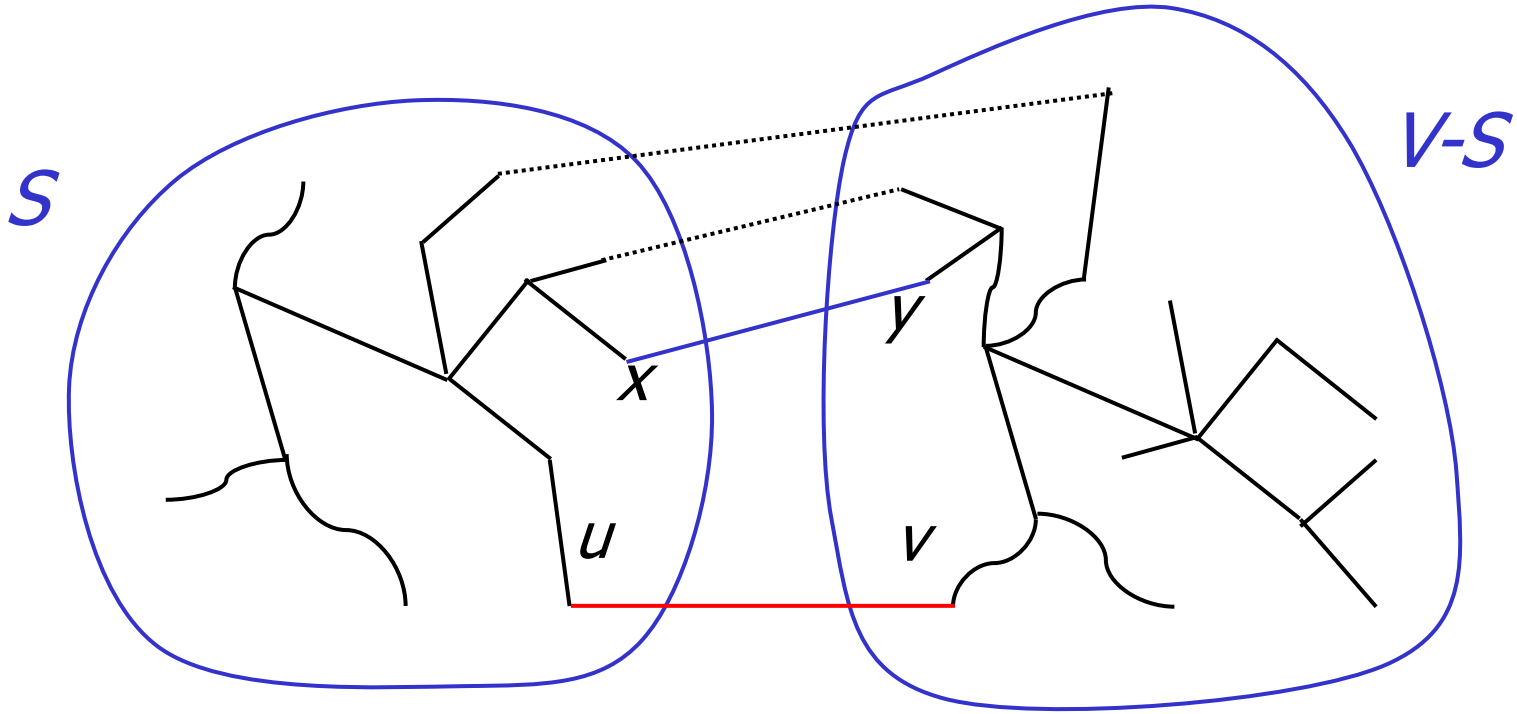
- A remoção de algum arco (u,v) particiona T *em* T_1 e T_2
- Como $w(T) = w(u,v) + w(T_1) + w(T_2)$
- T_1 é uma AEM de $G_1=(V_1,E_1)$
- T_2 é uma AEM de $G_2=(V_2,E_2)$



Escolha Gulosa

- Propriedade:
  - Escolhas localmente ótimas ou gulosas (greedy) levam á uma solução global ótima
- Teorema
  - Seja $G=(V, E)$, e seja $S \subseteq V$
  - Seja (u,v) a aresta de menor custo em G que conecta S a V – S
  - Então $(u,v) \in$ *a alguma AEM de G*

Escolha Gulosa (2)



Escolha Gulosa (3)

- Prova
  - Suponha $(u,v) \notin T$
  - Mesmo assim, u e v devem estar em T e deve haver um caminho de u até v em T
  - Seja qualquer aresta (x, y) neste caminho que cruza de S para $V - S$
  - Como (u,v) tem o menor custo, então $w(u,v) \leq w(x,y)$
  - Assim, a árvore que inclui (x,y) não pode ter o custo menor que árvore que inclui (u,v) e portanto ela não é mínima.

Algoritmo Genérico para AEM

```
AEM(G,w)
1 A←∅  // Conterá os arcos da AEM
2 enquanto A não formar uma AEM faça
3     Encontre um arco (u,v) seguro para A
4     A←A∪{(u,v)}
5 retorne A
```

Arco seguro – arco que garante que A:

- forma uma árvore
- inclui os arcos mínimos

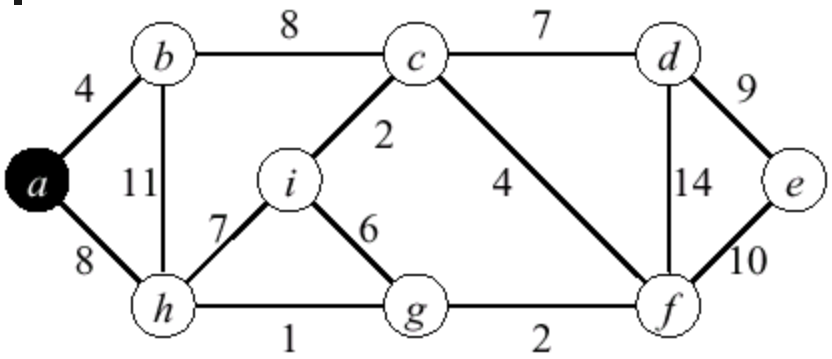
Alg. Genérico para AEM (2)

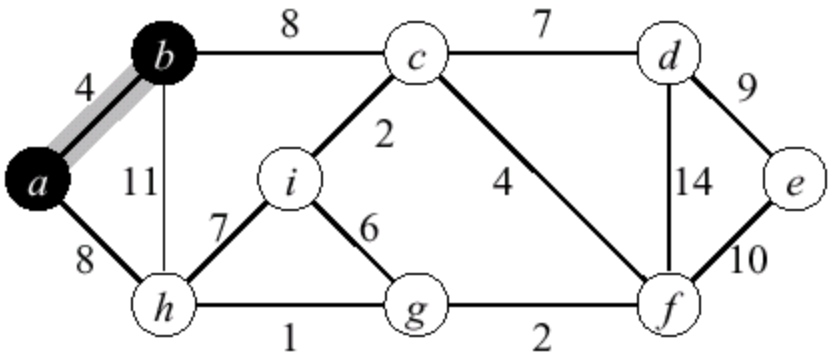
```
AEM2(G, w)
1    A←∅  // Conterá as arestas da AEM
2    enquanto A não for uma árvore espalhada faça
3.1     Faça um corte(S, V-S) em G que respeita A
3.2     Seja (u,v) a aresta mínima entre S e V-S
4     A←A∪{(u,v)}
5  retorne A
```

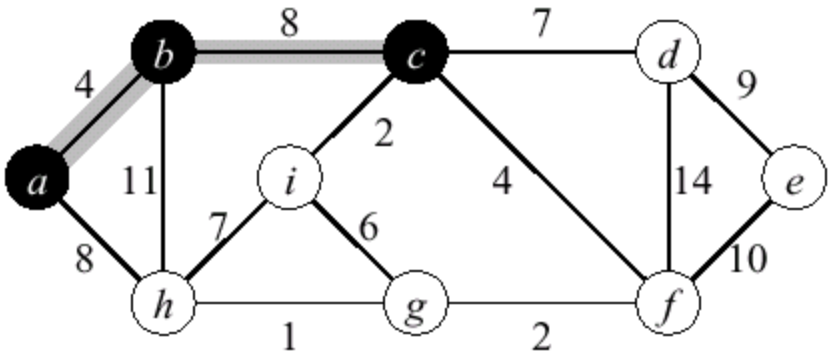
Algoritmo de Prim

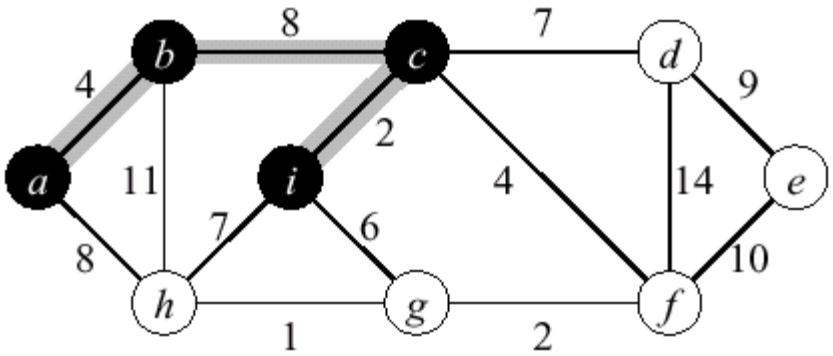
- Algoritmo baseado em vértices
- Constrói uma árvore T, com um vértice de cada vez.
- Utiliza um conjunto de vértices que corresponde a porção de T que já foi computada
- Rotula os vertices que estão fora deste conjunto com
  - *key*[*v*] – arco de menor custo que conecta *v* a um vértice no conjunto
  - *key*[*v*] = ∞, se não existe tal arco

Prim: exemplo

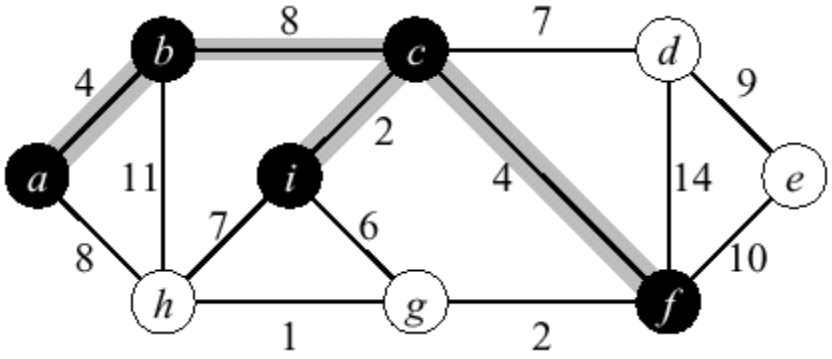


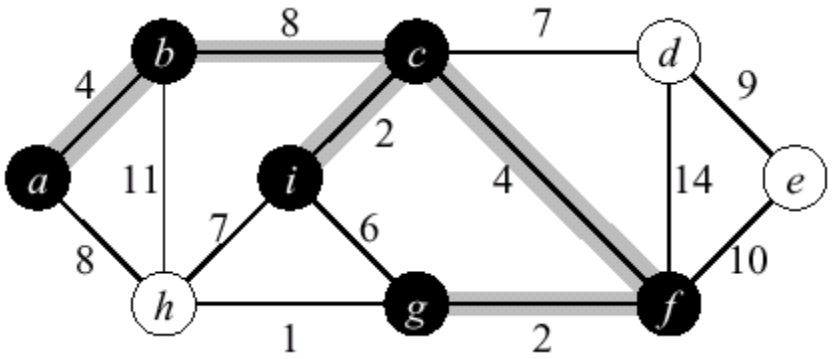


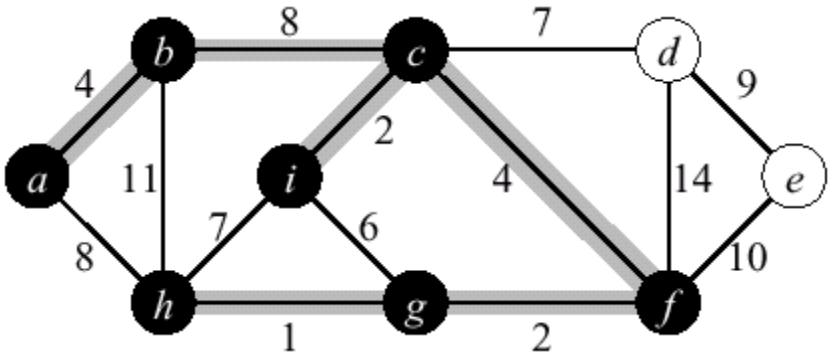


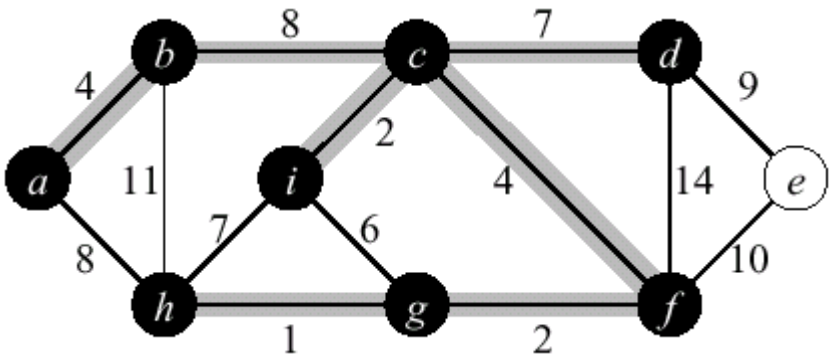


Prim: exemplo (2)

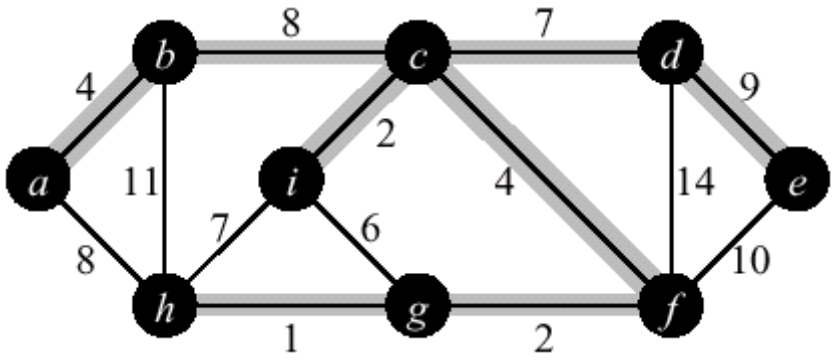








Prim: exemplo (3)



Algoritmo de Prim (2)

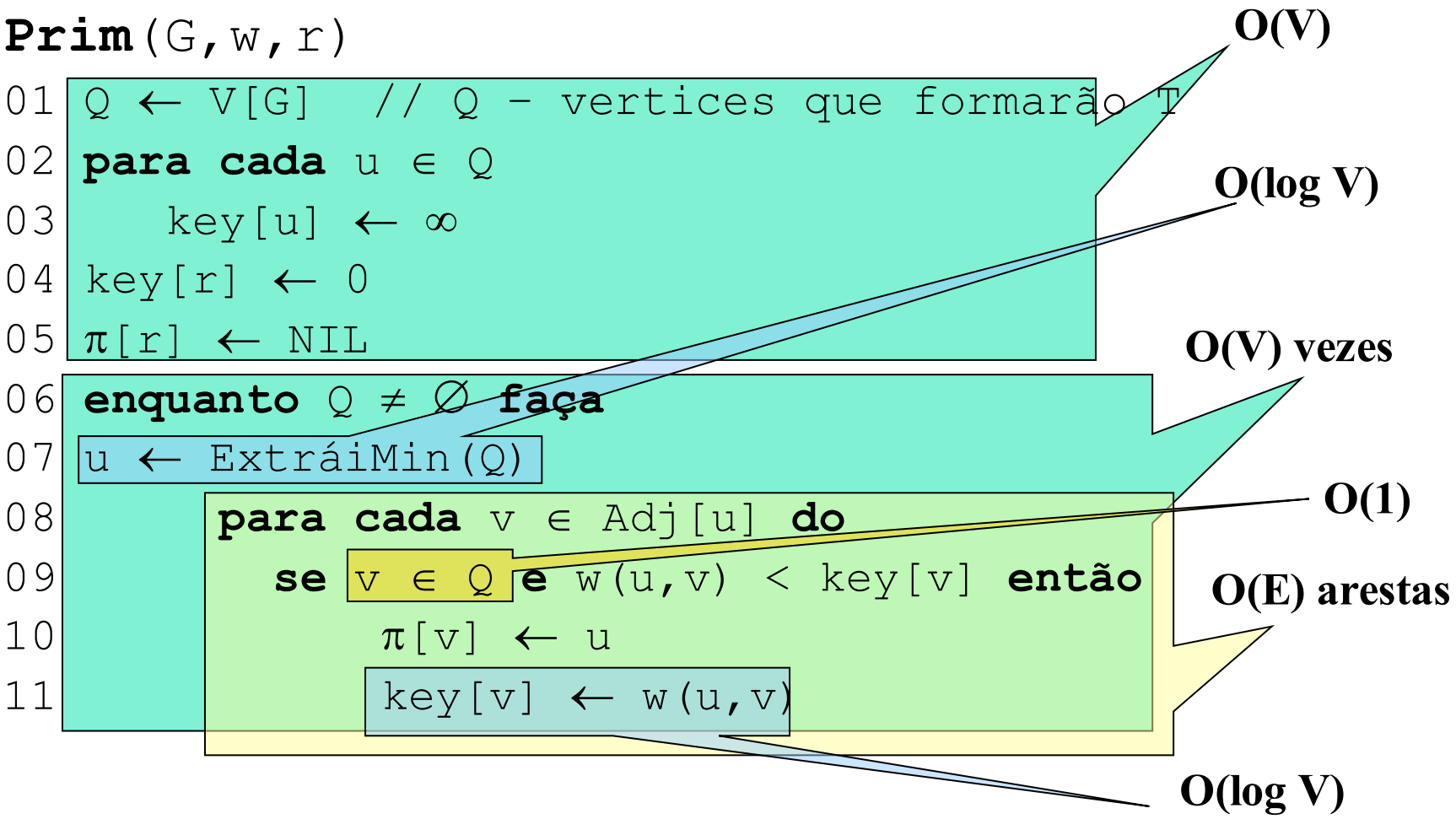
```
Prim(G,w,r)
01 Q ← V[G]  // Q - vertices que formarão T
02 para cada u ∈ Q
03    key[u] ← ∞
04 key[r] ← 0
05 π[r] ← NIL
06 enquanto Q ≠ ∅ faça
07 u ← ExtráiMin(Q)
08      para cada v ∈ Adj[u] do
09        se v ∈ Q e w(u,v) < key[v] então
10            π[v] ← u
11            key[v] ← w(u,v)
```

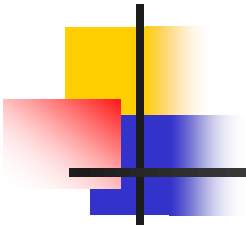
Algoritmo de Prim (3)

```
Prim(G,w,r)
01 Q ← V[G]  // Q - vertices que formarão T
02 para cada u ∈ Q
03     key[u] ← ∞
04 key[r] ← 0
05 π[r] ← NIL
06 enquanto Q ≠ ∅ faça
07 u ← ExtráiMin(Q)
08       para cada v ∈ Adj[u] do
09           se v ∈ Q e w(u,v) < key[v] então
10               π[v] ← u
11               key[v] ← w(u,v)
```

Atualiza key[v]

Algoritmo de Prim (4)





Filas de Prioridade

- Uma fila de prioridades (**FP**) é uma ED que mantém um conjunto S de elementos, cada um associado a uma chave (key).
- Uma FP deve suportar as seguintes operações:
  - ConstróiFP(S): Carrega a FP com os elementos de S
  - ExtráiMin(S): retorna e remove o elemento de S que tem a menor chave
  - Atualiza(S,x,novachave): muda a chave do elemento x
- Um **heap** binário pode ser usado
  - ConstróiFP – $O(n)$
  - ExtraíMin e Atualiza – $O(\lg n)$

Heap Binário

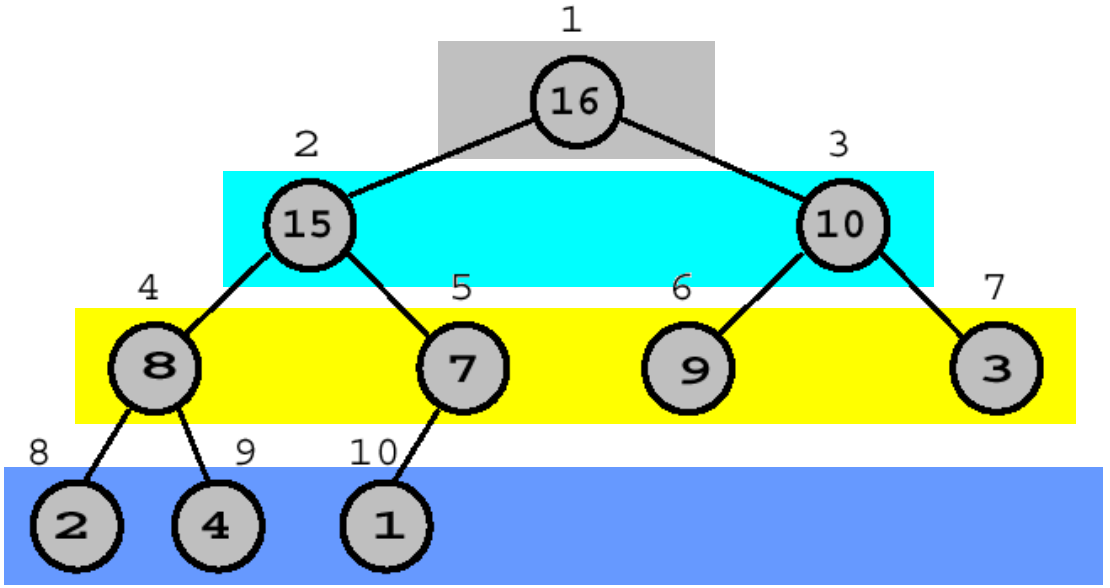
pai (*i*)
retorne $\lfloor i/2 \rfloor$

fesq (*i*)
retorne $2i$

fdir (*i*)
retorne $2i+1$

Propriedade:

$A[\text{Pai}(i)] \geq A[i]$



1	2	3	4	5	6	7	8	9	10
16	15	10	8	7	9	3	2	4	1

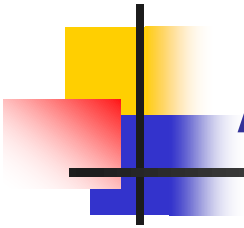
Nível: 3 2 1 0

*Williams, J. W. J. (1964), "Algorithm 232 - Heapsort", Communications of the ACM, 7 (6): 347–348, doi:10.1145/512274.512284

Prim: Tempo de Execução

- $|V|.T(\text{ExtráiMin}) + O(E).T(\text{Atualiza})$
- $O(V \lg V + E \lg V) = O(E \lg V)$

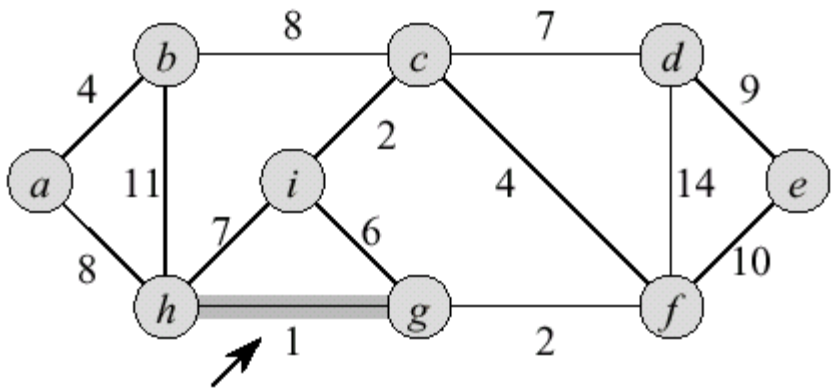
Fila	T(ExtráiMin)	T(Atualiza)	Total
array	$O(V)$	$O(1)$	$O(V^2)$
Heap binário	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Heap de Fibonacci	$O(\lg V)$	$O(1)$	$O(V \lg V + E)$

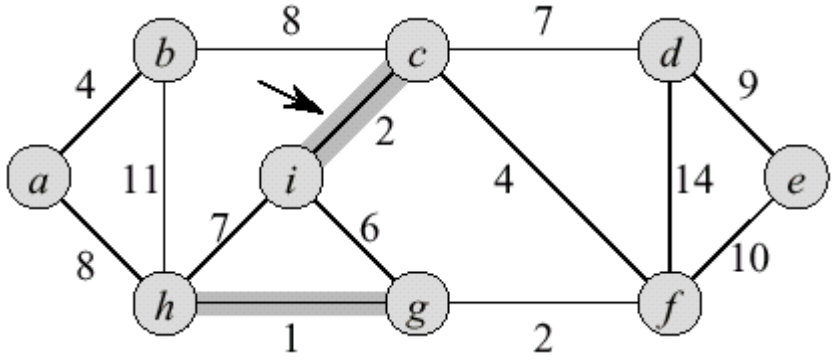


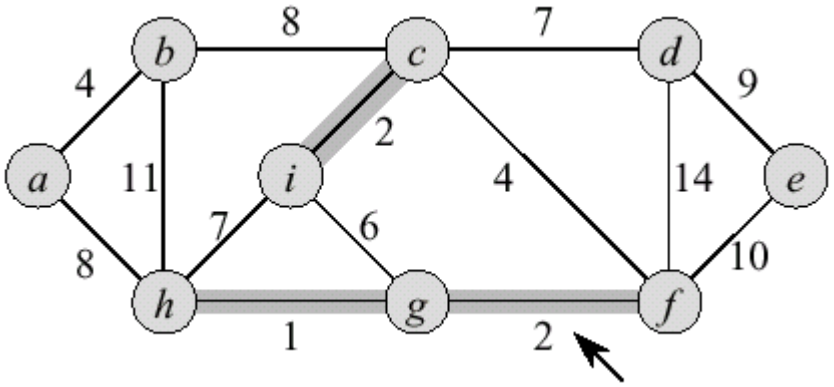
Algoritmo de Kruskal

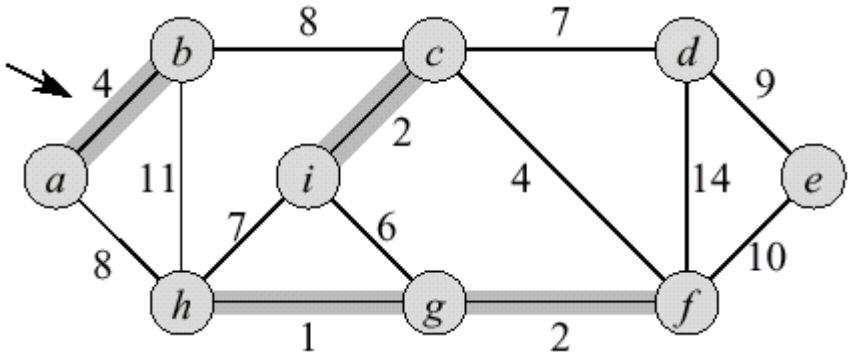
- Algoritmo baseado em arcos
- Adiciona um arco de cada vez na ordem crescente dos custos
- Mantém uma **floresta** *A*.
- Um arco é incluído se ele conecta vértices de árvores distintas de A.

Exemplo – Kruskal

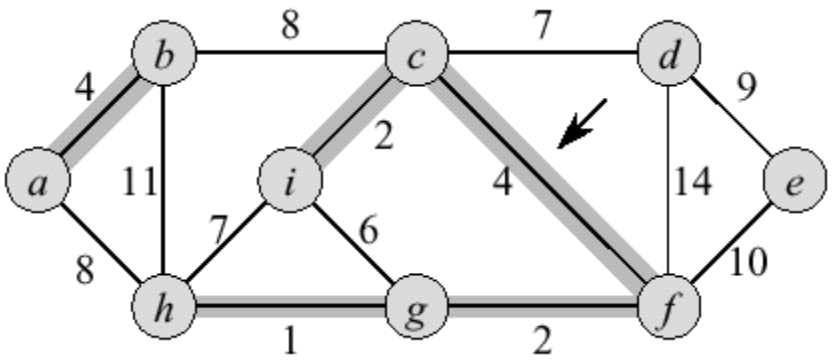


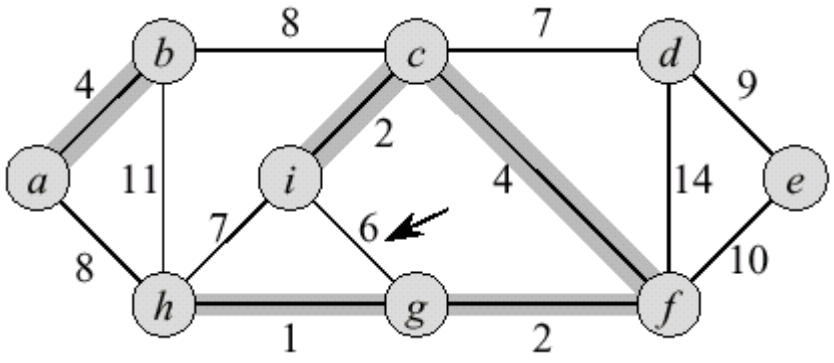


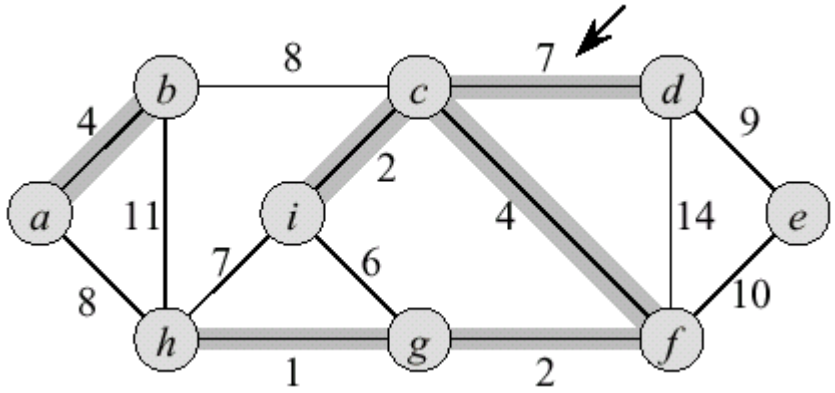


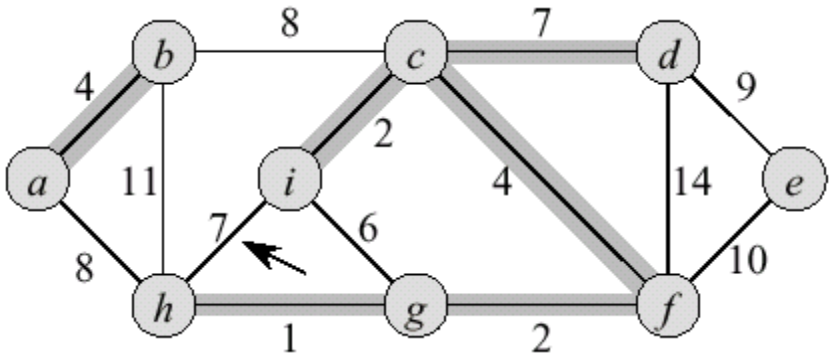


Exemplo – Kruskal (2)

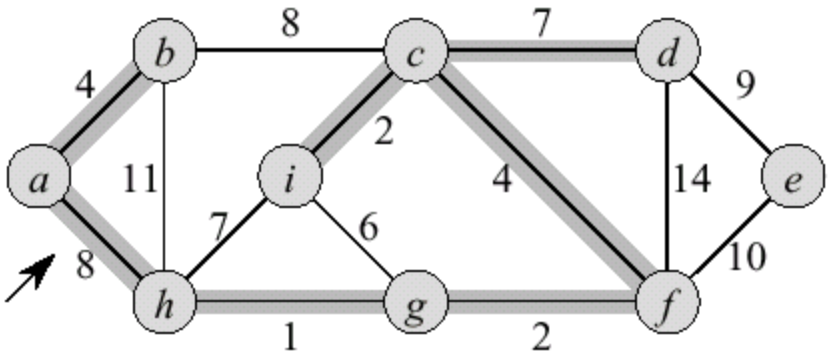


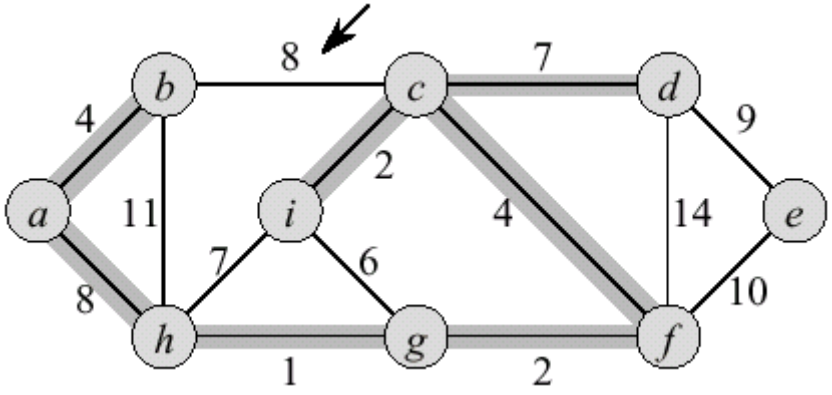


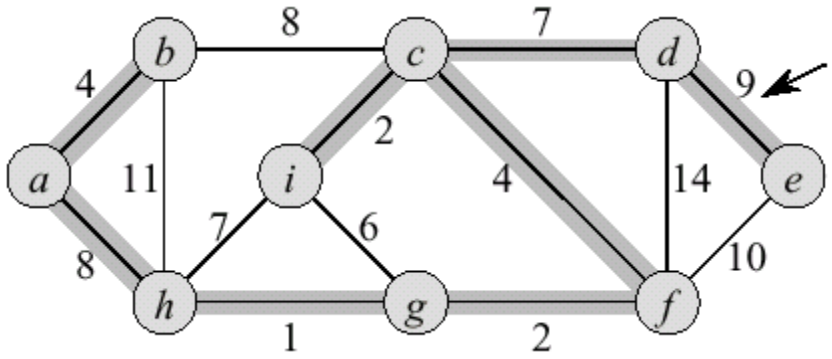


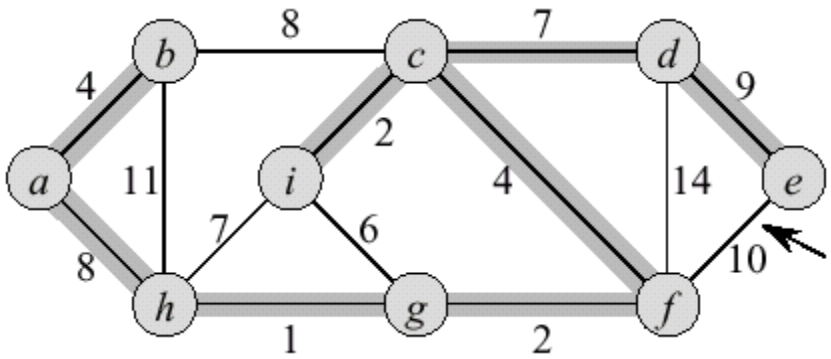


Exemplo – Kruskal (3)

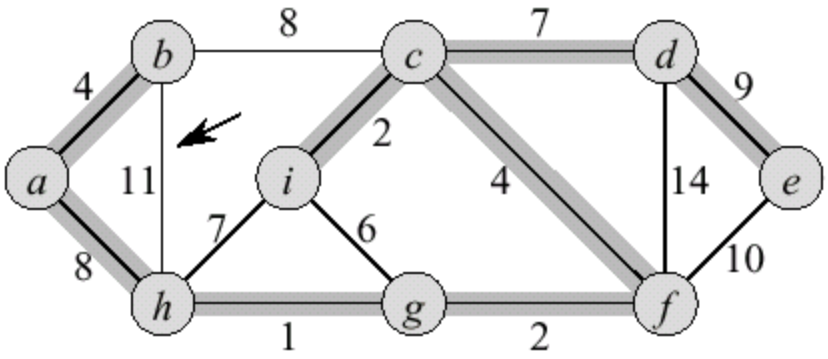


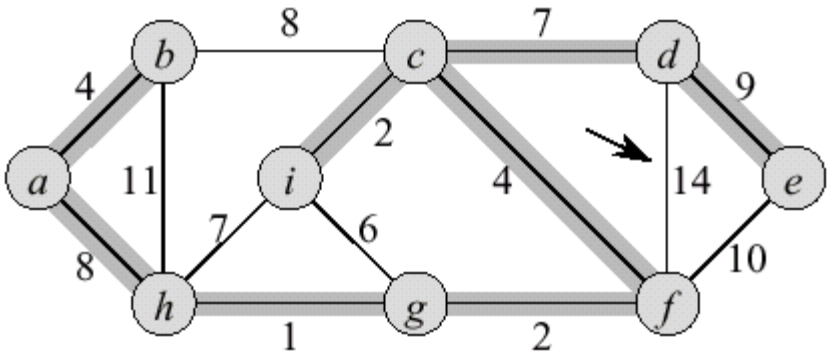






Exemplo – Kruskal (4)





Algoritmo de Kruskal (2)

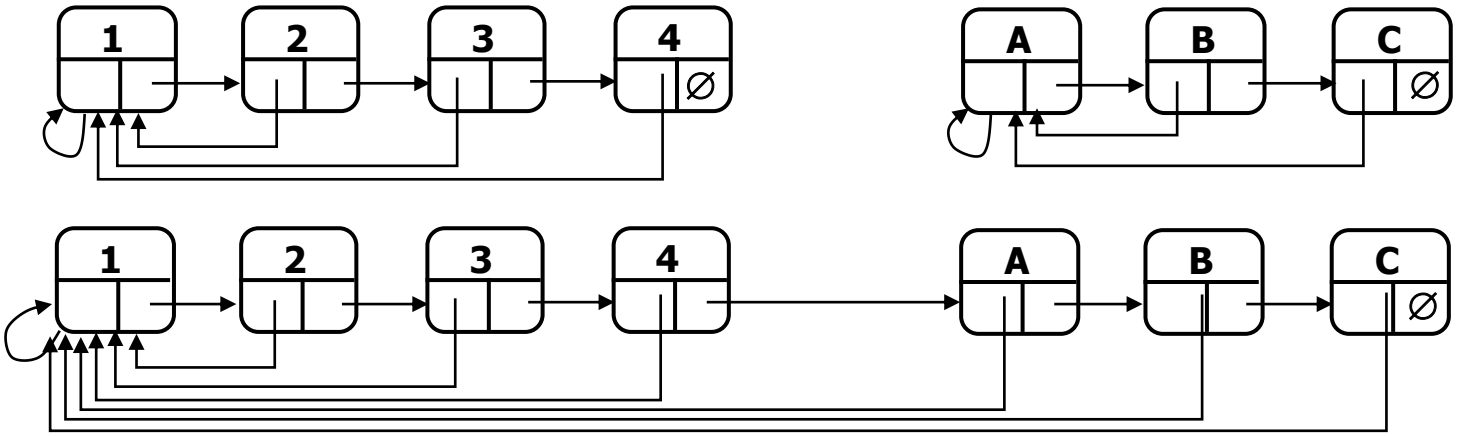
```
Kruskal(G,w)  /* G=(V,E)  */
01 A ← ∅
02 para cada vértice v ∈ V[G] do
03    Constrói-Conjunto({v})
04 Ordenar as arestas de acordo com o custo w
05 para cada arco (u,v) nesta ordem faça
06   se Conjunto_de(u) ≠ Conjunto_de(v) então
07      A ← A ∪ {(u,v)}
08      União(Conjunto_de(u),Conjunto_de(v))
09 retorne A
```

Implementação de Conjuntos

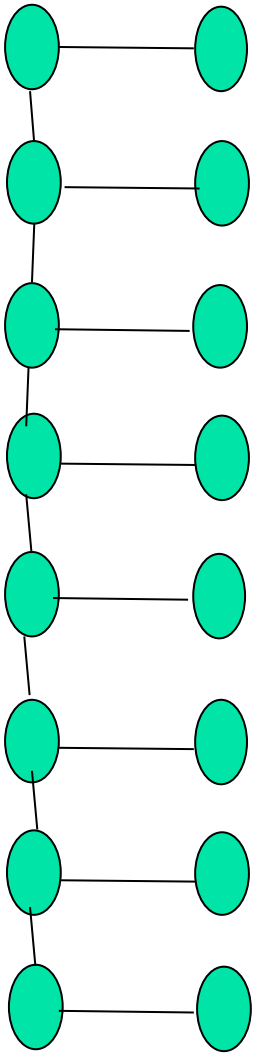
- Para implementar Kruskall é necessário utilizar uma lista de dados para manter conjuntos disjuntos de vértices
- Operações
  - Constrói_Conjunto(x): $S \leftarrow \{x\}$
  - União(S_i,S_j): $S \leftarrow S - \{S_i,S_j\} \cup \{S_i \cup S_j\}$
  - Conjunto_de(x): retorna S_i, tal que $x \in S_i$

Conjuntos disjuntos como listas

- Cada conjunto – lista de elementos identificados pelo primeiro elemento.
- Todos os elementos na lista apontam para o primeiro elemento
- União – adiciona a lista menor à lista maior
- Conjunto-de: $O(1)$, União(u,v): $O(\min\{|C(u)|, C(v)\})$

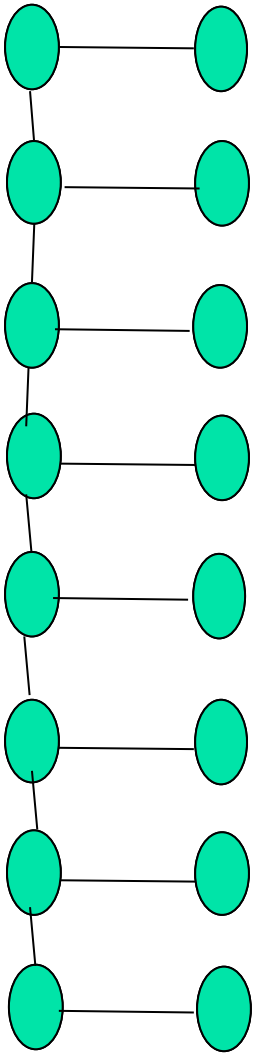


Custo das uniões
Melhor caso



- Temos ao todo n-1 uniões (número de arestas de uma AEM)
- Cada união tem custo igual ao menor conjunto unido
- Melhor caso, o menor conjunto sempre tem 1 elemento
- Custo total das uniões v-1, O(v)

Custo das uniões
Pior caso



- Ainda temos ao todo n-1 uniões (número de arestas de uma AEM)

- Mas cada união entre conjuntos de tamanho igual
- No exemplo:
- 8x1=8
- 4x2=8
- 2x4=8
- 1x8=8
- 8xlog(16)=8x4 = 32

- Custo total das uniões: v/2 log v, O(vlog v)

Kruskal: Tempo de Execução

- Inicialização: $O(V)$
- $\Theta(E \lg E) \sim \Theta(E \lg V)$
- $O(E)$ chamadas para Conjunto_De
- Custo de uniões
  - Seja $t(v)$ o número de vezes em que um vértice v é movido para um novo conjunto maior
  - Cada vez que é um vértice é movido, o novo conjunto tem seu tamanho pelo menos dobrado: $t(v) \leq \log V$
  - Tempo total de união $\sum_{v \in V} t(v) \leq |V| \log |V|$
- Tempo total: $O(E \lg V)$

Caminhos Mínimos

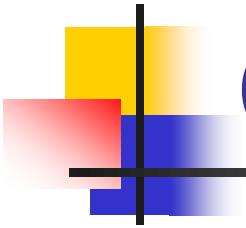


Caminho Mínimo

- Seja um digrafo (grafo dirigido) $G = (V,E)$ com uma função de custo $W: E \rightarrow R$
- O custo do caminho $p = v_1 \rightarrow v_2 \rightarrow \ldots \rightarrow v_k$ é

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$

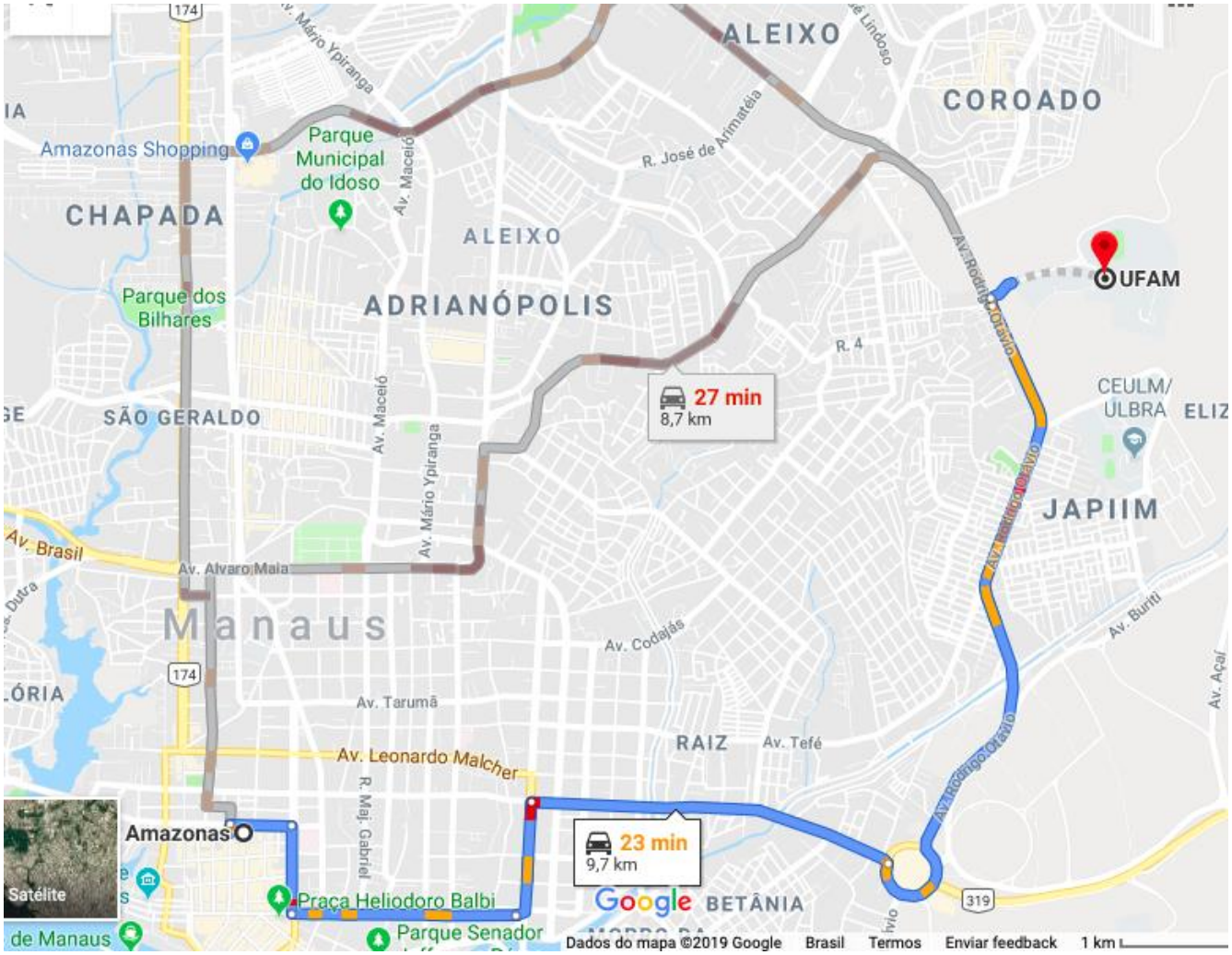
- Caminho mínimo: caminho de menor custo

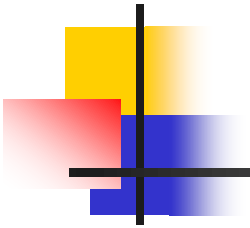


Caminhos Mínimos

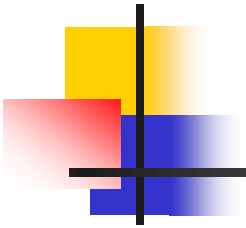
- Problemas de caminho mínimo
  - **Fonte Única (Destino Único).** Encontrar o caminho mínimo de um dado vértice (fonte) para todos os outros vértices.
  - **Par Único.** Dados dois vértices, achar o menor caminho entre eles. A solução para o problema de fonte única serve para este problema.
  - **Todos os Pares.** Encontrar o caminho mínimo entre todos os pares de vértices do grafo. Aplica-se **programação dinâmica**.

Aplicação Clássica





- Também é base para alguns algoritmos da área de redes
- Aplicado também em redes definidas por software para encontrar a topologia da rede.



Procurando por referências...

- Chuzhoy, Julia, and Sanjeev Khanna. "A new algorithm for decremental single-source shortest paths with applications to vertex-capacitated flow and cut problems. *ACM SIGACT*. 2019.
- Mozes, Shay, Yahav Nussbaum, and Oren Weimann. "Faster shortest paths in dense distance graphs, with applications." *Theoretical Computer Science* 711 (2018): 11-35.
- Sun, Yunchuan, et al. "Discovering time-dependent shortest path on traffic graph for drivers towards green driving." *Journal of Network and Computer Applications* 83 (2017): 204-212.

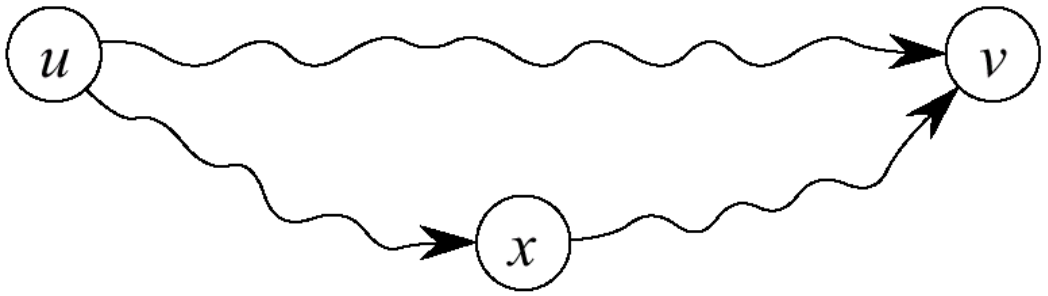
Sub-estrutura ótima

- **Teorema**: os sub-caminhos de um caminho mínimo são caminhos mínimos
- **Prova** (cut and paste)
  - Se algum sub-caminho não fosse mínimo, ele poderia ser substituído por um menor que levaria ao caminho mínimo total.



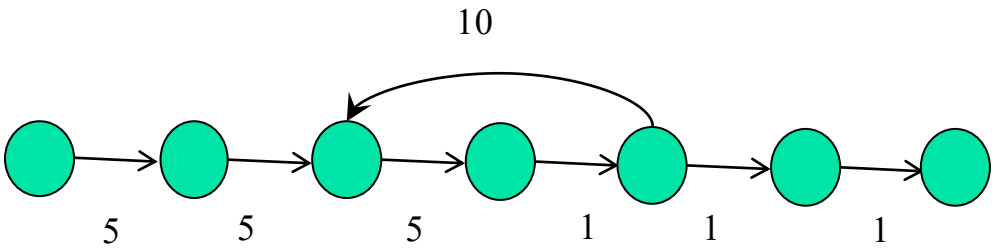
Inequação de triângulos

- Definição
  - $\delta(u,v) \equiv$ custo cam. min. entre *u-v*
- Teorema
  - $\delta(u,v) \leq \delta(u,x) + \delta(x,v)$ para qq *x*
- Prova
  - o cam. min. entre u-v não pode ser maior que nenhum outro caminho entre u-v, em particular o caminho que concatena os caminhos mínimos entre u-x e x-v



Ciclos

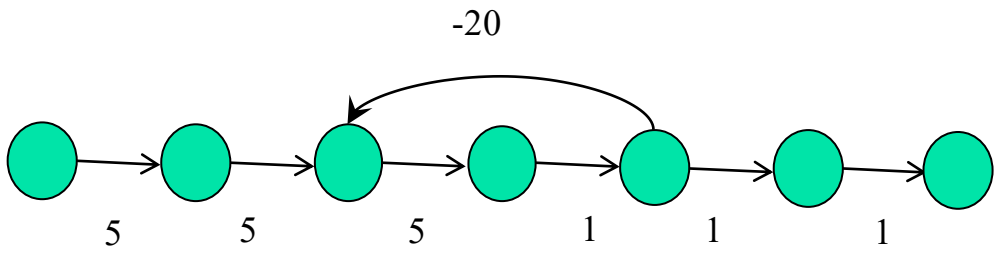
- Em geral, caminhos mínimos não podem ter ciclos, pois se tivessem os custo poderia ser reduzido removendo o ciclo.
- Qualquer caminho mínimo em um grafo não pode ter mais que $n - 1$ arestas, onde n é o nr. de vértices



18 sem o ciclo. 34 com o ciclo!

Pesos negativos e ciclos

- Arestas com pesos negativos podem ser consideradas no caminho mínimo, no entanto, neste caso, ciclos poderiam levar a caminhos de custo arbitrariamente menores

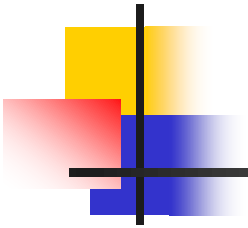


18 sem o ciclo. 4 com uma volta no ciclo!

-10 com 2 voltas, mas diminui a cada volta!



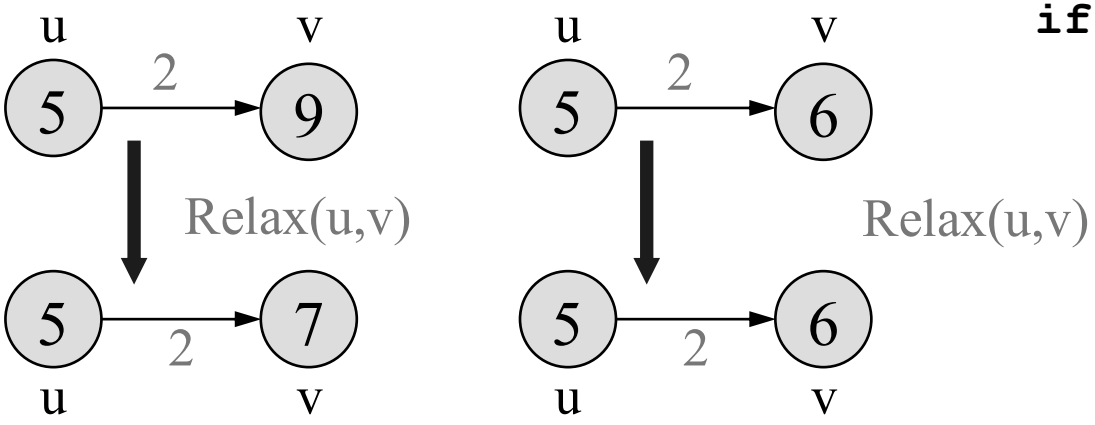
Antes de falar dos algoritmos em si, vamos falar de relaxamento...



Relaxamento é uma operação básica usada nos dois algoritmos clássicos para caminhos mínimos.

Relaxamento

- Para cada vértice no grafo, é mantido um valor d[v], uma estimativa do peso total do camin. Este valor é inicializado como ∞
- Relaxar um arco (u,v) consiste em testar se é possível melhorar o caminho corrente para v passando por u




```
Relax (u,v,w)
if d[v]>d[u]+w(u,v) then
    d[v] ← d[u]+w(u,v)
    π[v] ← u
```

Algoritmo de Dijkstra.



Pesquisador Holandês, viveu de 1930 a 2002.

Dijkstra, Edsger W. "A note on two problems in connexion with graphs." *Numerische mathematik* 1.1 (1959): 269-271.

Citado mais de 25 mil vezes na literatura!

Algoritmo de Dijkstra

- Arestas com pesos não negativos
- Algoritmo guloso similar à busca em largura
- Usa uma fila de prioridade Q usando $d[v]$ como chave
- Idéia básica
  - Mantêm um conjunto S de vértices já processados
  - A cada passo, seleciona o vértice u mais próximo, inclui o vértice u, adiciona-o a S e relaxa todos os arcos que saem de u.

Dijkstra

Q : Fila de Prioridade que contém os vértices e tem d[v] como chave

$Q = \{(v_1,d[v_1]), (v_2,d[v_2]),..., (v_V,d[v_V])\}$

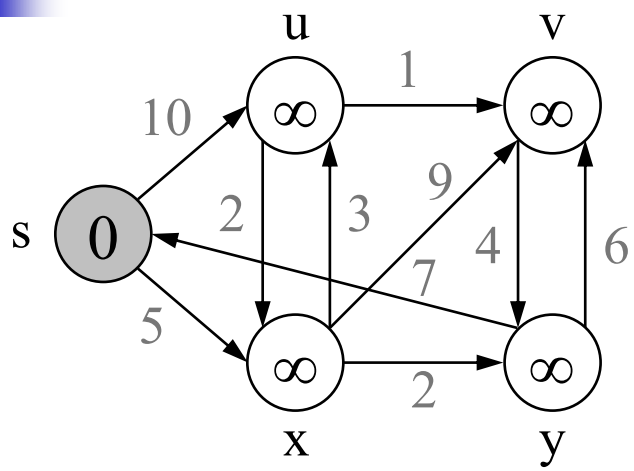
```
Dijkstra(G,w,s)
01 para cada v ∈ V
02    d[v] ← ∞
03 d[s]← 0
04 S ← ∅
05 Q ← V
06 enquanto Q ≠ ∅ faça
07    u ← ExtráiMin(Q)
08    S ← S ∪ {u}
09    para cada v ∈ Adj[u] faça
10          se d[v] > d[u]+w(u,v) então
11             d[v] ← d[u]+w(u,v)
```

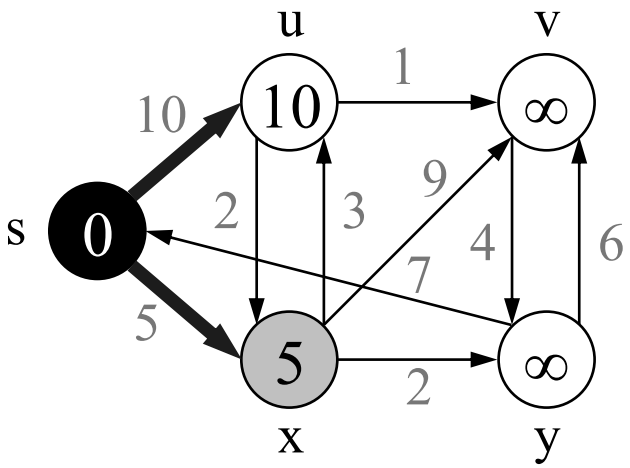
Conjunto de vértices já processados

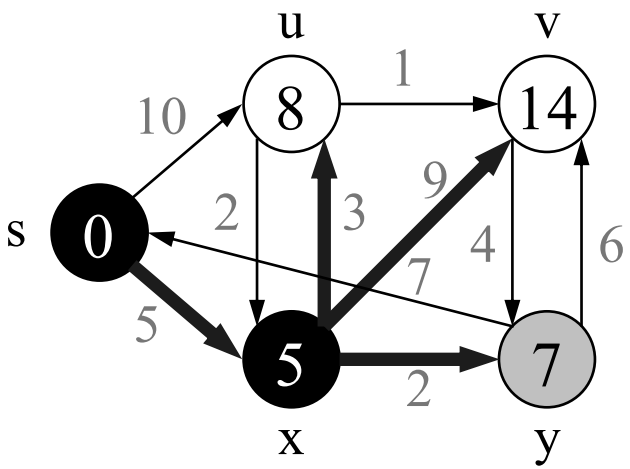
Retira o u de menor d[u] de Q

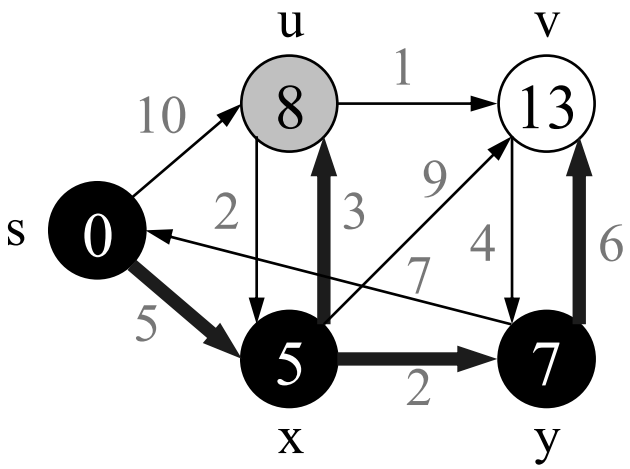
Relaxamento

Dijkstra: Exemplo

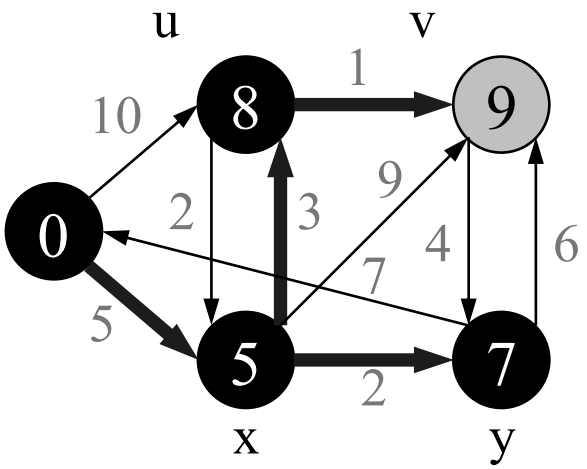


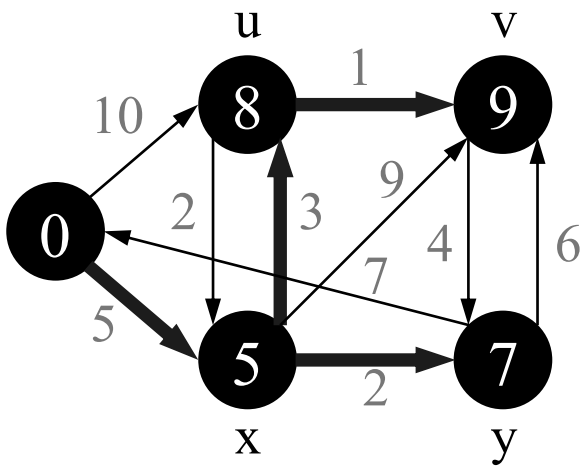






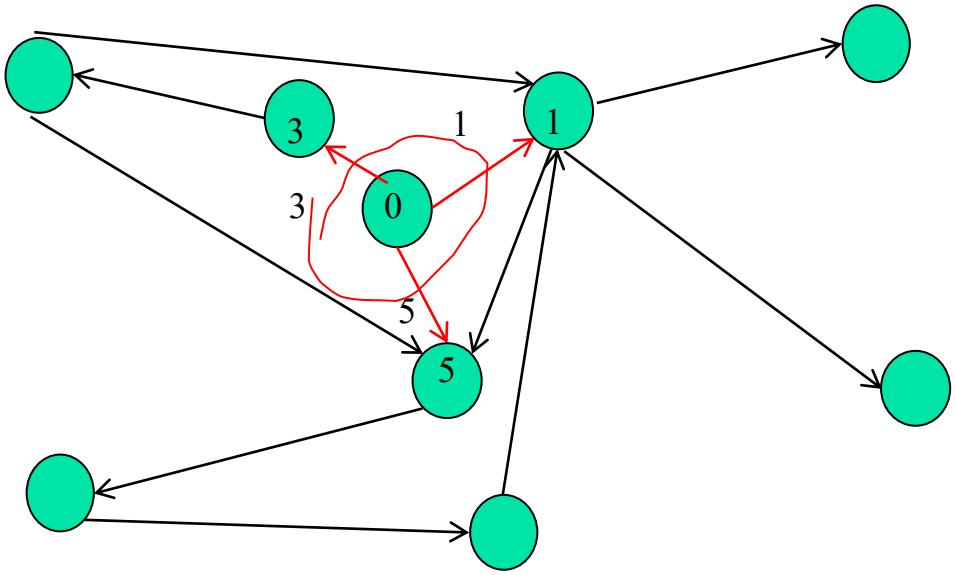
Dijkstra: Exemplo





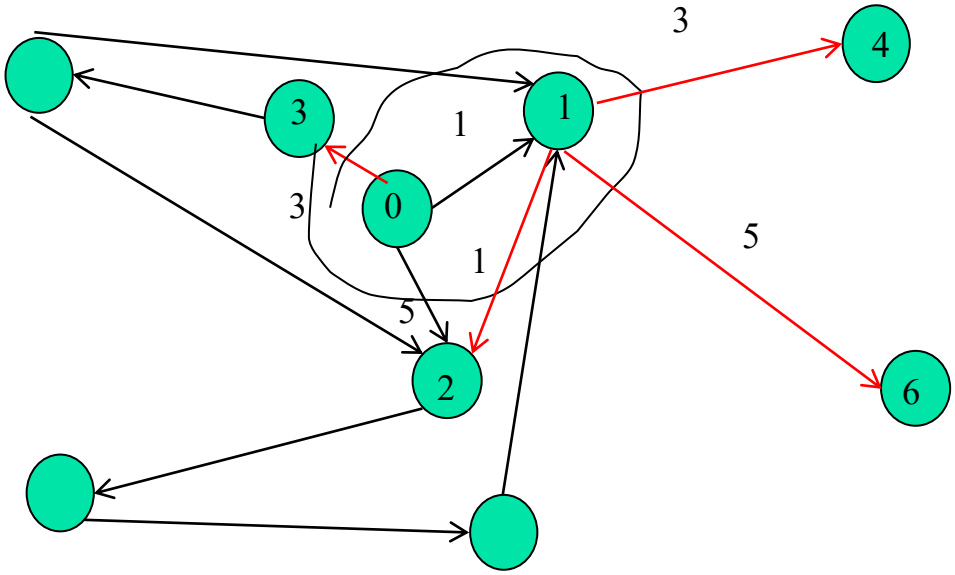
Por que funciona ?

- Menor dos caminhos com 1 aresta a partir da origem é um caminho mínimo



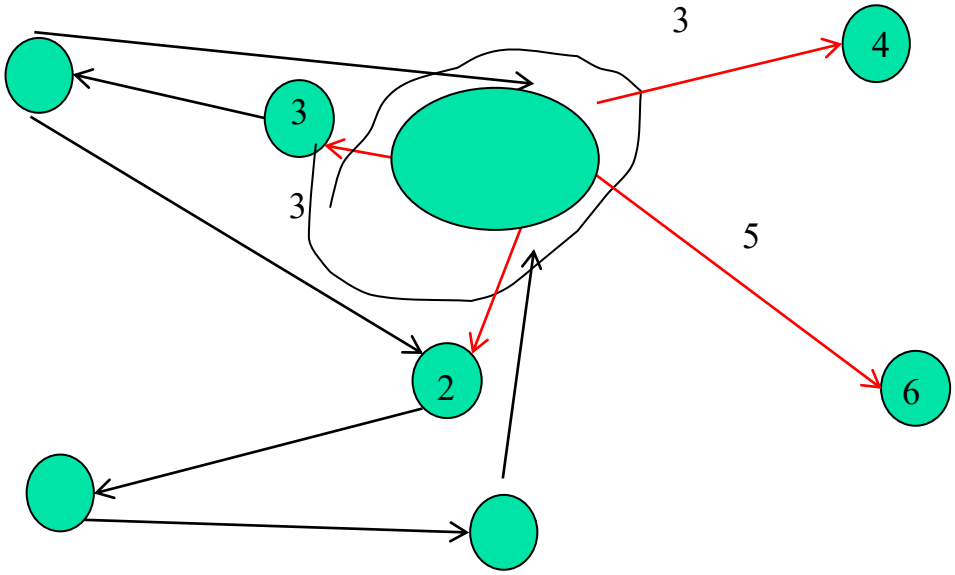
Por que funciona ?

- Passo seguinte tem o menor dos caminhos de tamanho 1 e 2 a partir da origem.



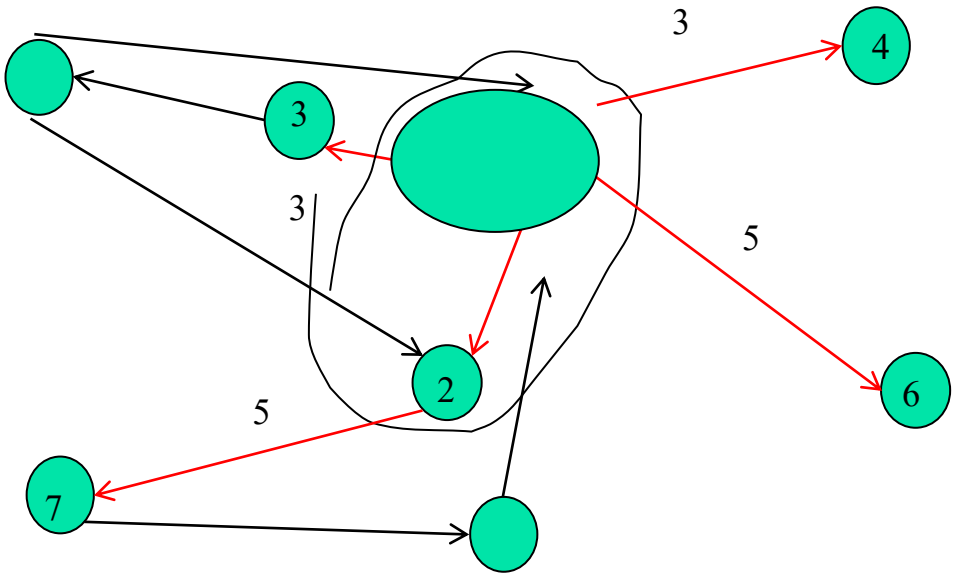
Por que funciona ?

- Passo seguinte tem o menor dos caminhos de tamanho 1 e 2 a partir da origem.



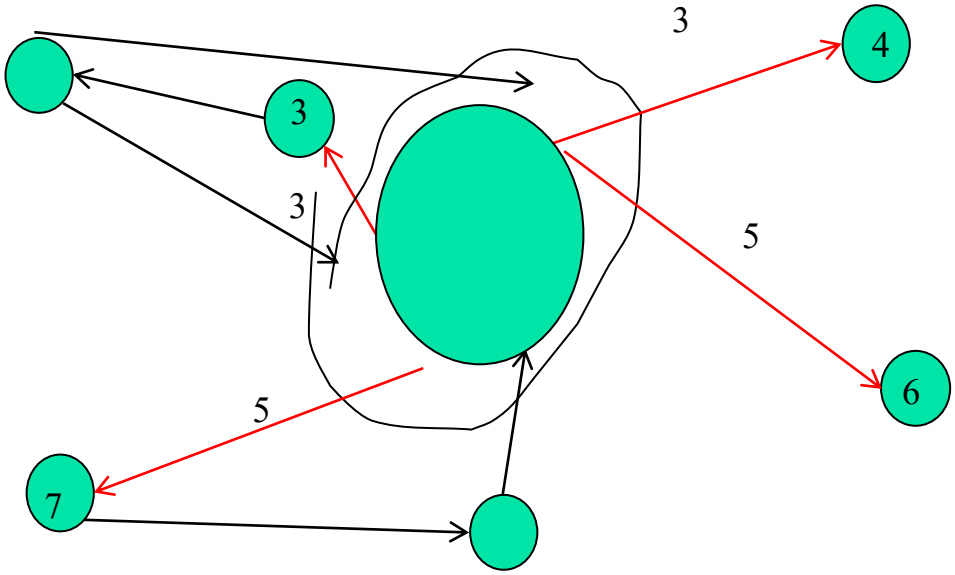
Por que funciona ?

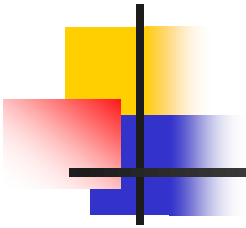
- Passo seguinte tem o menor dos caminhos de tamanho 1 e 2 a partir da origem.



Por que funciona ?

- Passo seguinte tem o menor dos caminhos de tamanho 1 e 2 a partir da origem.

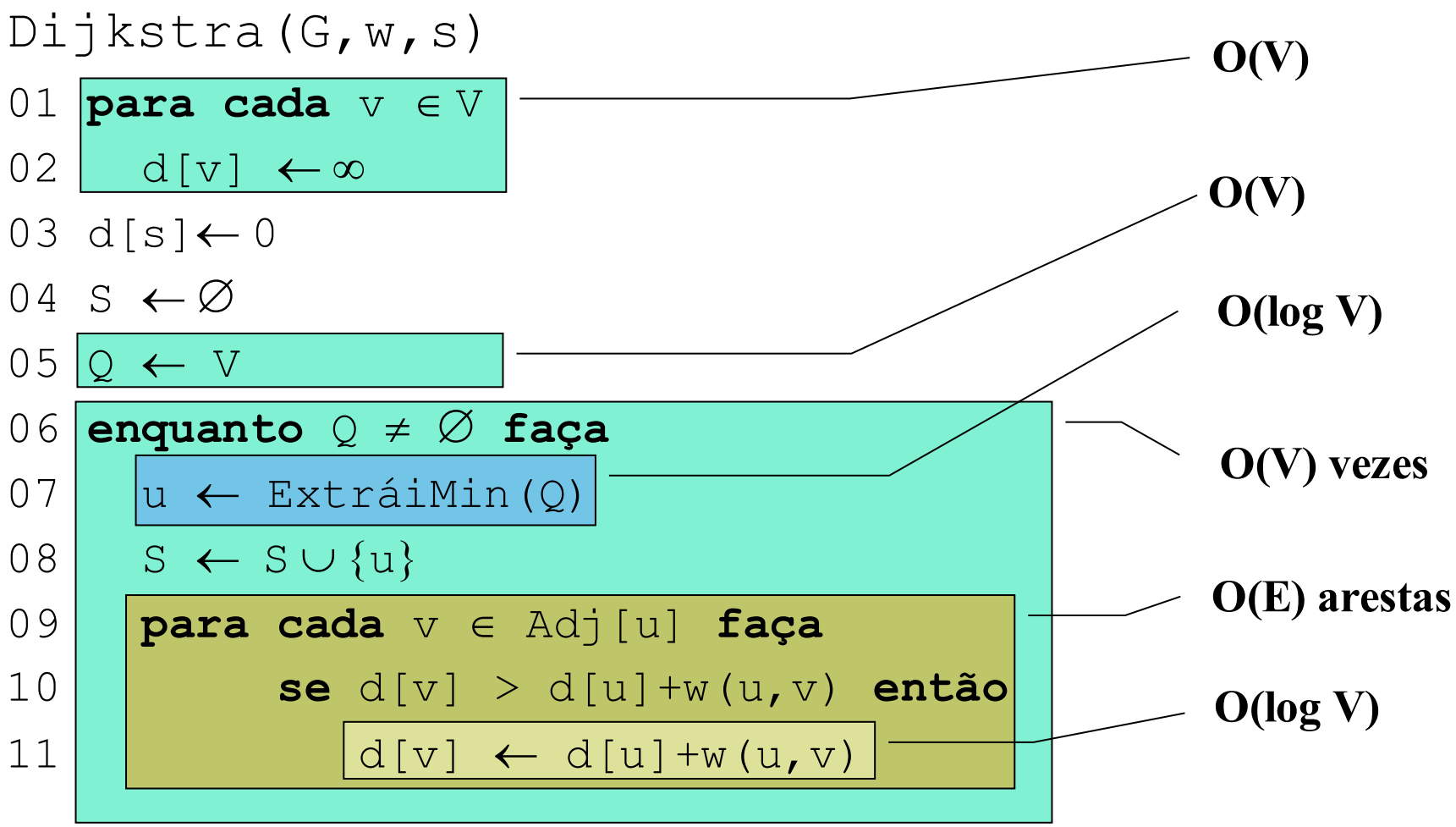




Por que funciona ?

- A cada passo, tenho um corte no grafo contendo S vértices
- A menor saída com 1 aresta entre o corte e o resto do grafo é um caminho mínimo!
- Porque qq outro caminho de uma aresta seria maior, e com mais arestas os caminhos só aumentam!

Dijkstra: Tempo de Execução



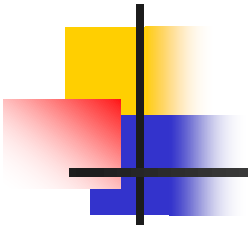
Dijkstra: Tempo de Execução

- Extract-Min: tempo $|V|$
- Decrementos de chaves: tempo $|E|$
- Time = $|V|\ T_{\text{Extract-Min}} + |E|\ T_{\text{decrementa-chave}}$
- Depende da implementação

Q	T(Extract-Min)	T(Decrease-Key)	
array	$O(V)$	$O(1)$	$O(V^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$	$O(1)$	$O(V \lg V + E)$

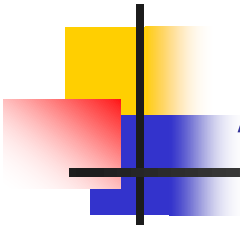
O que andam citando sobre... ?

- Qing, Guo, Zhang Zheng, and Xu Yue. "Path-planning of automated guided vehicle based on improved Dijkstra algorithm." *2017 29th Chinese control and decision conference (CCDC)*. IEEE, 2017.
- Hougardy, Stefan, Jannik Silvanus, and Jens Vygen. "Dijkstra meets Steiner: a fast exact goal-oriented Steiner tree algorithm." *Mathematical Programming Computation* 9.2 (2017): 135-202.



Bellman-Ford

Proposto por: Alfonso Shimbel (1955)
Publicado por: Richard Bellman(1958), Lester Ford Jr.(1956) e por Edward F. Moore(1957)
Alguns chamam de Bellman-Ford Moore.
Poderia ser chamado de Shimbel ???



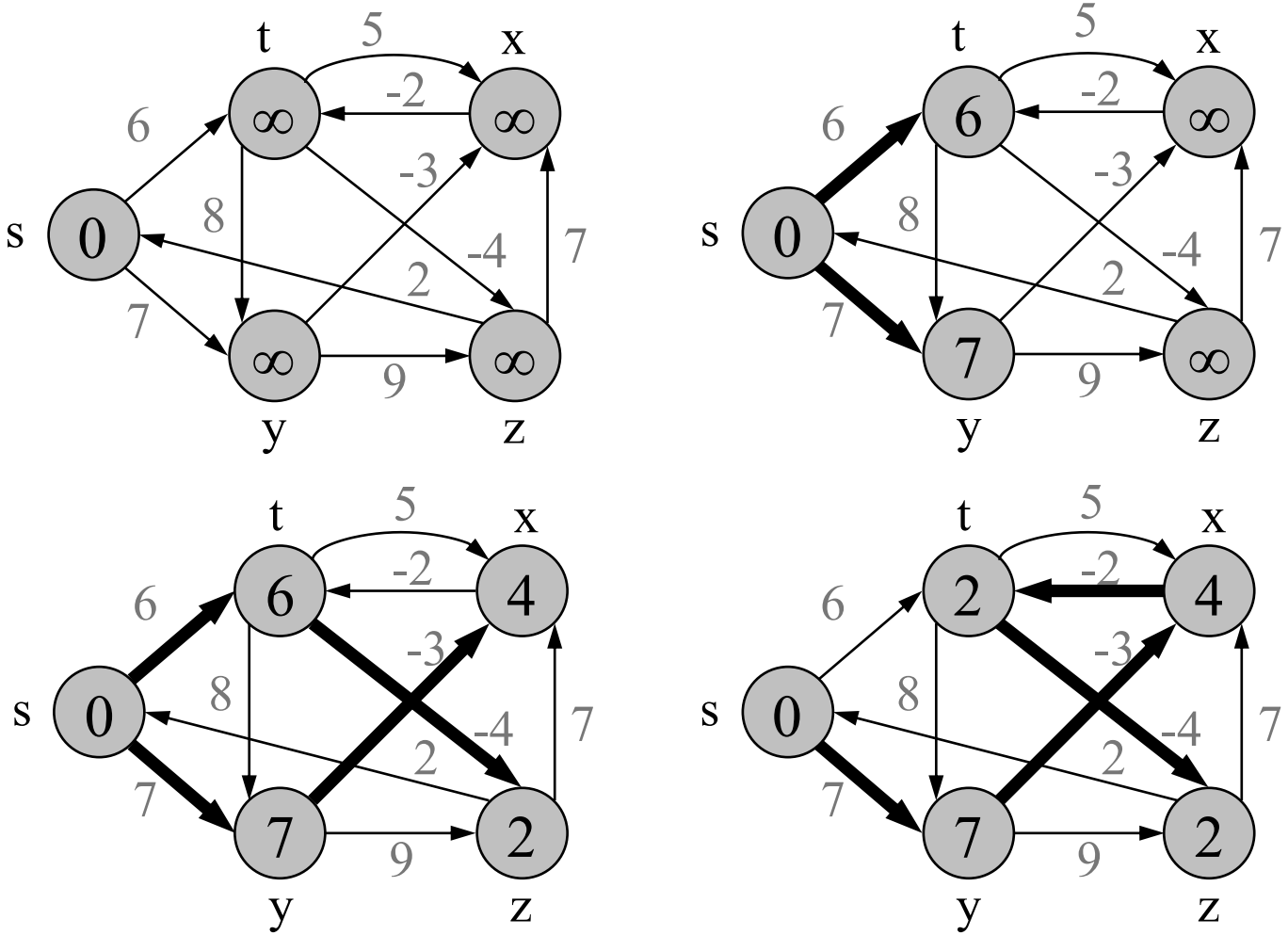
Algoritmo de Bellman-Ford

- Dijkstra não funciona com arestas negativas
  - Não se pode assumir que o custo dos caminhos só podem aumentar.
- O Algortimo de **Bellman-Ford** detecta ciclos negativos ou retorna os caminhos mínimos.

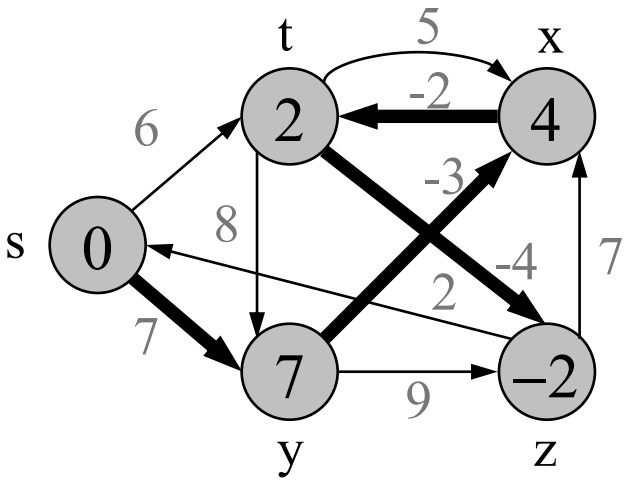
Bellman-Ford

```
Bellman-Ford(G,w,s)
01 para cada v ∈ V[G]
02     d[v] ← ∞
03 d[s] ← 0
04 π[r] ← NIL
05 para i ← 1 até |V[G]|-1 faça
06     para cada aresta (u,v) ∈ E[G] faça
07         Relax (u,v,w)
08 para cada aresta (u,v) ∈ E[G] faça
09     se d[v] > d[u] + w(u,v) então retorne falso
10 retorne verdadeiro
```

Bellman-Ford: Exemplo



Bellman-Ford: Exemplo



Bellman-Ford: Tempo

```
Bellman-Ford(G,w,s)
01 para cada v ∈ V[G]
02     d[v] ← ∞
03 d[s] ← 0
04 π[r] ← NIL
05 para i ← 1 até |V[G]|-1 faça
06     para cada aresta (u,v) ∈ E[G] faça
07         Relax (u,v,w)
08 para cada aresta (u,v) ∈ E[G] faça
09     se d[v] > d[u] + w(u,v) então retorne falso
10 retorne verdadeiro
```

$$(|V|-1)|E| + |E| = \Theta(VE)$$

Bellman-Ford: Corretude

- Provar que se o menor caminho entre s e u tem i arestas, então depois do i-ésimo passo do algoritmo teremos o menor caminho, ou seja, $d[u]=\delta(s,u)$
- Seja $\delta_i(s,u)$ o custo do caminho de s até u que é o mínimo entre todos os caminhos que contém no máximo i arestas

Bellman-Ford: Corretude (2)

- Provar por indução que $d[u] = \delta_i(s,u)$ depois da i-ésima iteração
  - Base da indução. Inicio do algoritmo
  - Hipótese da indução: $d[u] = \delta_{i-1}(s,u)$
  - Passo indutivo
    - Caso 1: $\delta_i(s,u) = \delta_{i-1}(s,u) \Rightarrow d[u] = \delta_i(s,u)$
    - Caso 2: $\delta_i(s,u) = \delta_{i-1}(s,z) + w(z,u)$
      - Na iteração cada arco é relaxado, inclusive (z,u), então $d[u] = \delta_i(s,u)$

Belman-Ford: Corretude (3)

- Se |V|=*n*, temos que depois de *n-1* iterações: $d[u] = \delta_{n-1}(s,u)$, para cada vértice *u*.
- Se ainda existe alguma aresta a relaxar no grafo então ainda existe um vértice *u* tal que $\delta_n(s,u) < \delta_{n-1}(s,u)$.
- Mas só existem *n* vértices. Portanto deve haver um ciclo, ele deve ser negativo.
- Caso contrário, $d[u]= \delta_{n-1}(s,u) = \delta(s,u)$, para todo *u*, uma vez que qualquer caminho mínimo terá nó máximo *n-1* arestas.

O que andam citando sobre eles ?

- Dinitz, Yefim, and Rotem Itzhak. "Hybrid Bellman–Ford–Dijkstra algorithm." *Journal of Discrete Algorithms* (2017)
- Mo, Yuanqiu, Soura Dasgupta, and Jacob Beal. "Robustness of the Adaptive Bellman–Ford Algorithm: Global Stability and Ultimate Bounds." *IEEE Transactions on Automatic Control* (2019)
- Daggitt, Matthew L., Alexander JT Gurney, and Timothy G. Griffin. "Asynchronous convergence of policy-rich distributed bellman-ford routing protocols." *Proceedings ACM SIG on Data Communication*. 2018.

- Kannan, Vidhya, and Saraswathi Appasamy. "Employing the Bellman-Ford Algorithm with Score Functions to Address the Linear Diophantine Fuzzy Shortest Path Problem in Network Analysis." *Mathematical Modelling of Engineering Problems* 10.5 (2023).

- Tejaswini, Pallapothala, Rajesh Doriya, and Kaushlendra Sharma. "Efficient Mobile Robot Navigation with D* Lite and Bellman Ford Hybrid Algorithm." *Proceedings of the 2023 6th International Conference on Information Science and Systems*. 2023.

- Chauhan, Shishir Singh, and Dilip Kumar. "Mode Search Optimization Algorithm for Traffic Prediction and Signal Controlling Using Bellman–Ford with TPFN Path Discovery Model Based on Deep LSTM Classifier." *SN Computer Science* 4.5 (2023): 686.



- Timofeeva, Olga, et al. "Modification of the Bellman–Ford Algorithm for Finding the Optimal Route in Multilayer Network Structures." *Computation* 11.4 (2023): 74.

- **Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, Longhui Yin (2025): „Breaking the Sorting Barrier for Directed Single-Source Shortest Paths“. 57th Annual ACM Symposium on Theory of Computing (STOC). https://dl.acm.org/doi/10.1145/3717823.3718179**